COMPUTER ENGINEERING

QUEZON CITY

| | | | |
|---|---|---|---|
| **COURSE CODE/TITLE:** | CPE 010 | **SECTION:** | CpE21S3 |
| **DATE PERFORMED:** | 07/16/2026 | **DATE SUBMITTED:** | 07/21/2026 |
| **NAME:** | Wayne Kaisen C. Paraiso | **INSTRUCTOR:** | Engr. Jimlord Quejado |

## Hand-on Activity 2.1 Arrays, Pointers, and Dynamic Memory Allocation

### 1. PROCEDURE OUTPUT

Screenshot

final.cpp > Student > Student(const Student &)

```cpp
#include <iostream>
#include <string.h>

class Student{
    private:
        std::string studentName;
        int studentAge;

    public:
        //constructor
        Student(std::string newName ="John Doe", int newAge=18){
            studentName = std::move(newName);
            studentAge = newAge;
            std::cout << "Constructor Called." << std::endl;
        };

        //deconstructor
        ~Student(){
            std::cout << "Destructor Called." << std::endl;
        }

        //Copy Constructor
        Student(const Student &copyStudent){
            std::cout << "Copy Constructor Called" << std::endl;
            studentName = copyStudent.studentName;
            studentAge = copyStudent.studentAge;
        }
        //Display Attributes
        void printDetails(){
            std::cout << this->studentName << " " << this->studentAge << std::endl;
        }
};
```

<table>
<tr><td></td><td>

```
34  int main(){
35
36      Student student1("Roman", 28);
37      Student student2(student1);
38      Student student3;
39      student3 = student2;
40
41      return 0;
42  }
```

```
Constructor Called.
Copy Constructor Called
Constructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
PS C:\Users\TIPQC\Documents\CPE 010 PARAISO\LAB2.2>
```

</td></tr>
<tr><td>Observation</td><td>For student1, it takes the two inputs and assigns them to their respective variables in the Student class, then prints out "Constructor Called." Next, it runs student2, which uses the copy constructor. student2 copies student1 with the help of the copy constructor function. When the copy constructor is called, it first prints out a message saying that the copy constructor was called, then it copies the variables from student1 and assigns them to its own variables. After that, it executes student3, which has no input. It then proceeds to the constructor function, which has a default newName of "John Doe" and a default age of 18, and assigns both default values to student3. Lastly, it executes student3 = student2, which takes the values of student2 and assigns them to student3's variables. After that, it deletes the objects in reverse order and frees up the memory occupied by the objects to make room for new ones.</td></tr>
</table>

Table 2.1 Inital Driver Program

<table>
<tr><td>Screenshot</td><td>

```
34  int main(){
35
36      const size_t j = 5;
37      Student studentList[j] = {};
38      std::string namesList[j] = {"Carly", "Freddy", "Sam", "Zack", "Cody"};
39      int ageList[j] = {15, 16, 18, 19, 16};
40
41      return 0;
42  }
```

</td></tr>
</table>

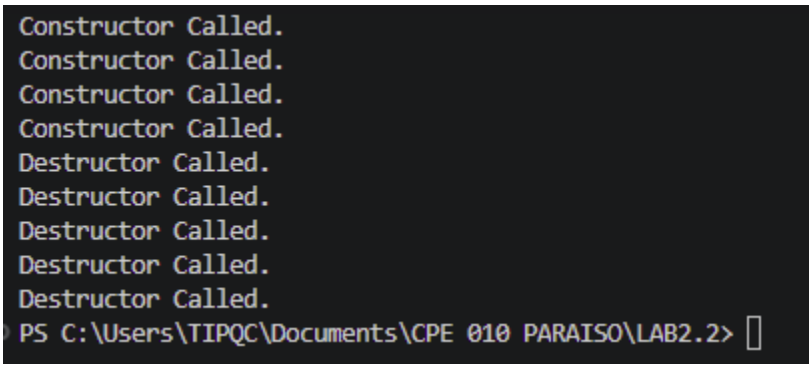
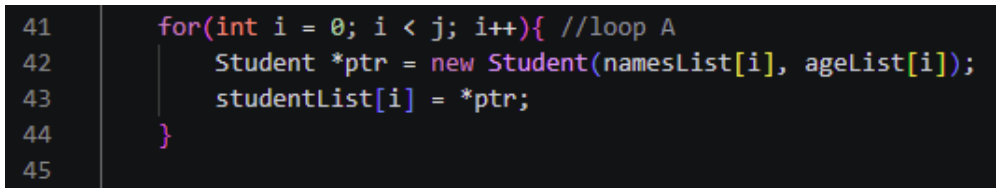
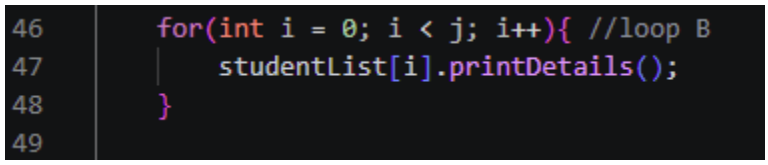
| | |
|---|---|
| |  |
| Observation | When it runs studentList, it goes to the constructor function and assigns the default student name, which is John Doe, to the nameList array. It then transfers it to the studentList array, where it is added to index 0. Next, it assigns the default age, which is 18, to the age variable at index 0. It repeats this process until the array is full and all five indexes have values. During each cycle, it prints out that the constructor function was called every time it finishes a cycle. After the loop, it moves to the next lines, which assign names to the nameList array and values to the ageList array. Finally, it runs return 0;, which then deletes the studentList array and frees up the memory occupied by it. |

Table 2.2 Modified Driver Program with Student List

| | |
|---|---|
| Loop A |  |
| Observation | This loop creates a new student object using the values at the current index of both the namesList and ageList arrays according to the value of i. It then creates a pointer variable and assigns the address of the student object to it according to the value of i, which corresponds to the index of the studentList array. |
| Loop B |  |
| Observation | This loop prints out the values of an index in an array which is studentList in this case. It takes the value of i and prints out the values of that index |

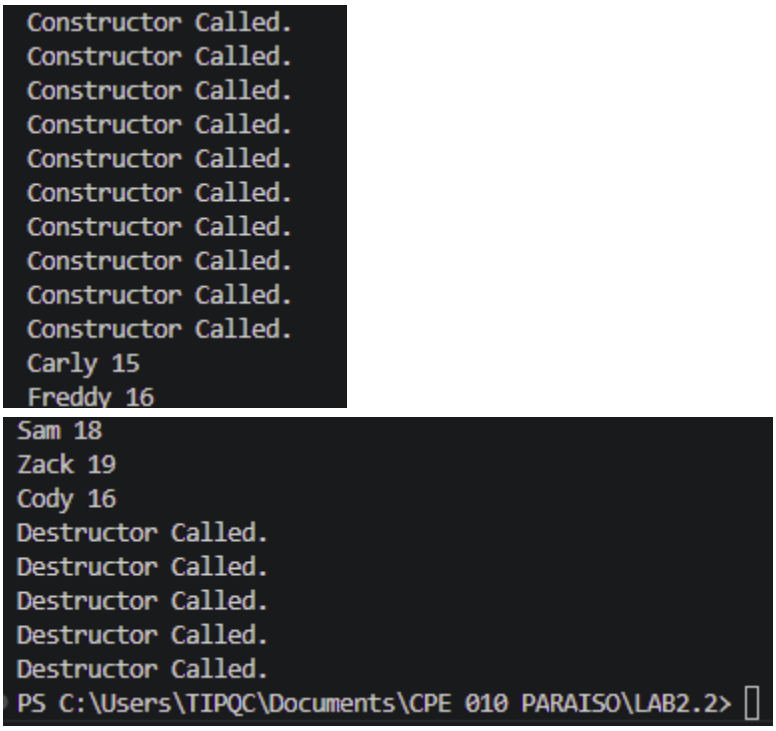
| | |
|---|---|
| Output |  |
| Observation | This correctly outputs what the code is intended to do. It first constructs the five student objects in the studentList array with their default values. Then, it assigns the values in the nameList array to their respective indexes, and does the same for the ageList array. After that, Loop A runs and creates a student object using the values from the nameList and ageList arrays, then uses a pointer to assign the object to the corresponding index in the studentList array based on the value of i. This process is repeated a total of five times. Next, Loop B runs and prints each index of the studentList array according to the value of i. Lastly, it runs the destructor function, which deletes the student objects and frees the memory occupied by each index of the studentList array. This also happens five times, once for each index in the array. |

Table 2.3 Final Driver Program

## 2. ANSWERS TO SUPPLEMENTAL ACTIVITY

supp1.cpp > main()

```cpp
#include <iostream>
#include <string>

class Fruit{
    private:
        std::string fruitName;
        int fruitPrice;
        int fruitQuantity;

    public:
        Fruit(std::string newName, int newPrice=0, int newQuantity=0){
                fruitName = std::move(newName);
                fruitPrice = newPrice;
                fruitQuantity = newQuantity;

                std::cout << "Constructor Called." << std::endl;
        }

        ~Fruit(){
            std::cout << "Destructor Called." << std::endl;
        }

        Fruit(const Fruit &copyFruit){
            std::cout << "Copy Constructor Called" << std::endl;
            fruitName = copyFruit.fruitName;
            fruitPrice = copyFruit.fruitPrice;
            fruitQuantity = copyFruit.fruitQuantity;
        }

        Fruit& operator=(const Fruit& other){
            std::cout << "Copy Assignment Operator Called" << std::endl;
```

```
32
33          if (this == &other)   // self-assignment check
34              return *this;
35
36          fruitName = other.fruitName;
37          fruitPrice = other.fruitPrice;
38          fruitQuantity = other.fruitQuantity;
39
40          return *this;
41      }
42
43      std::string GetName() const { return fruitName; }
44      int GetPrice() const { return fruitPrice; }
45      int GetQuantity() const { return fruitQuantity; }
46
47      int CalculateSum() const {
48          return fruitPrice * fruitQuantity;
49      }
50
51      void Display(){
52          std::cout << "Name: " << fruitName
53                    << ", Price: " << fruitPrice
54                    << ", Quantity: " << fruitQuantity
55                    << ", Subtotal: " << CalculateSum() << std::endl;
56      }
57
58  };
```

```cpp
class Vegetable{
    private:
        std::string vegName;
        int vegPrice;
        int vegQuantity;

    public:
        Vegetable(std::string newName ="", int newPrice=0, int newQuantity=0){
                vegName = std::move(newName);
                vegPrice = newPrice;
                vegQuantity = newQuantity;
                std::cout << "Constructor Called." << std::endl;
        }

        ~Vegetable(){
            std::cout << "Destructor Called." << std::endl;
        }

        Vegetable(const Vegetable &copyVeg){
            std::cout << "Copy Constructor Called" << std::endl;
            vegName = copyVeg.vegName;
            vegPrice = copyVeg.vegPrice;
            vegQuantity = copyVeg.vegQuantity;
        }

        Vegetable& operator=(const Vegetable& other){
            std::cout << "Copy Assignment Operator Called" << std::endl;
```

```
87
88              if (this == &other)   // self-assignment check
89                  return *this;
90
91              vegName = other.vegName;
92              vegPrice = other.vegPrice;
93              vegQuantity = other.vegQuantity;
94
95              return *this;
96          }
97
98          std::string GetName() const { return vegName; }
99          int GetPrice() const { return vegPrice; }
100         int GetQuantity() const { return vegQuantity; }
101
102         int CalculateSum(){
103             return vegPrice * vegQuantity;
104         }
105
106         void Display(){
107             std::cout << "Name: " << vegName << ", Price: " << vegPrice
108                       << ", Quantity: " << vegQuantity
109                       << ", Subtotal: " << CalculateSum() << std::endl;
110         }
111 };
```

```
};

int main(){

    const int fruitCount = 2;
    int vegCount = 2;


    Fruit** fruits = new Fruit*[fruitCount];
    fruits[0] = new Fruit("Apple", 10, 7);
    fruits[1] = new Fruit("Banana", 5, 8);

    Vegetable** vegetables = new Vegetable*[vegCount];
    vegetables[0] = new Vegetable("Broccoli", 15, 12);
    vegetables[1] = new Vegetable("Lettuce", 8, 10);


    std::cout << "\n--- Fruits ---\n";
        for (int i = 0; i < fruitCount; i++){
            fruits[i]->Display();
        }

    std::cout << "\n--- Vegetables ---\n";
        for (int i = 0; i < vegCount; i++){
            vegetables[i]->Display();
        }
```

```
137
138     int total = 0;
139     for (int i = 0; i < fruitCount; i++) total += fruits[i]->CalculateSum();
140     for (int i = 0; i < vegCount; i++) total += vegetables[i]->CalculateSum();
141     std::cout << "\nGrand Total: " << total << std::endl;
142
143
144     for (int i = 0; i < vegCount; i++){
145         if (vegetables[i]->GetName() == "Lettuce"){
146             delete vegetables[i];
147
148             for (int j = i; j < vegCount - 1; j++)
149                 vegetables[j] = vegetables[j+1];
150             vegCount--;
151             break;
152         }
153     }
154
155 }
```

## 3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

**Tools Used:** **ChatGPT, Claude**

*Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.*

**Purpose of Use:** **Grammar Correction, Code assistance**

*Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.*

**Extent of Use:** **Only fix the grammatical errors and without changing the concept of the text. I asked Claude for help with the errors I cant find in my code and give tips about what I should use.**

*Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.*

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.




Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.